

Auth Module

Role Module Auth Documentation

Base URL: `{{BASE_URL}}`

Default header: `Content-Type: application/json`

This document follows your real UI/API flow:

- register -> OTP verify -> selection step (Guru/NM) -> account setup/profile -> application submission -> pending approval -> login

Promoter Module

End-to-End Flow

1. `POST /api/auth/register` with role: "promoter"
2. `POST /api/auth/otp/verify` (token is issued here)
3. `GET /api/gurus/available` (select Guru screen)
4. `POST /api/auth/setup` (account setup/full name)
5. `POST /api/promoters/applications` (submit promoter application with selected Guru)
6. `GET /api/promoters/applications/me` (track status)
7. after approval, `POST /api/auth/login`
8. after login, `PATCH /api/auth/me` for profile updates

1) Register Promoter

API: `POST /api/auth/register`

Purpose: create promoter account and send OTP

Request

```
{
  "email": "promoter@example.com",
  "password": "Promoter@1234",
  "role": "promoter",
  "role_requested": "promoter",
  "referral_token": "OPTIONAL_REF_TOKEN"
}
```

Response (success shape)

```
{
  "error": false,
  "message": "Registration successful. Please verify your email with the OTP sent to your inbox.",
  "data": {
    "email": "promoter@example.com",
    "userId": 101,
    "accountStatus": "pending",
    "emailStatus": "pending",
    "roleRequested": "promoter",
    "otp": "123456",
    "challengeId": 98765,
    "expires-in": 600
  }
}
```

2) Verify OTP (Token Issued)

API: POST /api/auth/otp/verify

Purpose: verify OTP and generate accessToken/refreshToken

Request

```
{
  "userId": 101,
  "otp": "123456",
  "challengeId": 98765
}
```

Response (success shape)

```
{
  "error": false,
  "message": "Email verified successfully. You are now logged in.",
  "data": {
    "userId": 101,
    "accessToken": "eyJ...",
    "refreshToken": "eyJ...",
    "expires-at": "2026-04-17T12:00:00.000Z",
    "emailStatus": "verified",
    "setupRequired": true,
    "role": "promoter",
    "user": {}
  }
}
```

```
}  
}
```

3) Guru Selection

API: GET /api/gurus/available

Purpose: fetch available Gurus for promoter to choose from before application submit

4) Account Setup (after OTP/token)

API: POST /api/auth/setup

Header: Authorization: Bearer <accessToken>

Purpose: complete account setup (name)

Request

```
{  
  "fullName": "Promoter Full Name"  
}
```

Response

```
{  
  "error": false,  
  "message": "Your account has been set up successfully.",  
  "data": {  
    "user": {},  
    "setupCompleted": true  
  }  
}
```

5) Submit Promoter Application (selected Guru)

API: POST /api/promoters/applications

Header: Authorization: Bearer <accessToken>

Purpose: submit promoter application with selected Guru and agreements

Request

```
{  
  "guru_user_id": 55,
```

```
{
  "full_name": "Promoter Full Name",
  "avatar_url": "https://cdn.example/avatar.png",
  "agreed_to_terms": true,
  "agreed_to_promoter_agreement": true,
  "agreed_to_activation_fee_terms": true
}
```

Response

```
{
  "error": false,
  "message": "Your promoter application has been submitted successfully. Please wait for approval.",
  "data": {
    "application": {
      "id": 9001,
      "accountStatus": "pending",
      "guruId": 55,
      "territoryName": "London",
      "activationFee": 85,
      "activationFeeInPence": 8500,
      "invoiceId": 12001,
      "currency": "GBP"
    }
  }
}
```

6) Application Status + Post-login Profile Update

- GET /api/promoters/applications/me to track pending/approved state
- PATCH /api/auth/me after login to update profile (name, avatarUrl)

Guru Module

End-to-End Flow

1. POST /api/auth/register with role: "guru"
2. POST /api/auth/otp/verify (token issued)
3. GET /api/network-managers/available (select Network Manager)
4. POST /api/auth/setup
5. POST /api/gurus/applications (submit guru application)
6. GET /api/gurus/applications/me (status)
7. after approval, POST /api/auth/login

8. post-login PATCH /api/auth/me

1) Register Guru

API: POST /api/auth/register

Request

```
{
  "email": "guru@example.com",
  "password": "Guru@1234",
  "role": "guru",
  "role_requested": "guru"
}
```

Response (success shape)

```
{
  "error": false,
  "message": "Registration successful. Please verify your email with the OTP sent to your inbox.",
  "data": {
    "email": "guru@example.com",
    "userId": 202,
    "roleRequested": "guru",
    "accountStatus": "requested",
    "emailStatus": "pending",
    "otp": "654321",
    "challengeId": 77881,
    "expires-in": 600
  }
}
```

2) Verify OTP

API: POST /api/auth/otp/verify

Request

```
{
  "email": "guru@example.com",
  "otp": "654321"
}
```

Response (success shape)

```
{
  "error": false,
  "message": "Email verified successfully. You are now logged in.",
  "data": {
    "accessToken": "eyJ...",
    "refreshToken": "eyJ...",
    "setupRequired": true,
    "role": "guru",
    "user": {}
  }
}
```

3) Select Network Manager + Submit Guru Application

Select list API: GET /api/network-managers/available

Submit API: POST /api/gurus/applications

Submit request

```
{
  "network_manager_user_id": 33,
  "avatar_url": "https://cdn.example/avatar.png",
  "contract_name": "Guru Contract Name",
  "agreed_to_terms": true,
  "agreed_to_guru_agreement": true
}
```

Submit response

```
{
  "error": false,
  "message": "Your Guru application has been submitted successfully. Please wait for approval.",
  "data": {
    "application": {
      "id": 7001,
      "networkManagerId": 33,
      "networkManagerName": "Manager Name",
      "territoryName": "London",
      "contractName": "Guru Contract Name",
      "accountStatus": "pending",
      "createdAt": "2026-04-17T12:00:00.000Z"
    }
  }
}
```

4) Status + Profile Update

- GET /api/gurus/applications/me for pending/approved status
 - PATCH /api/auth/me after login (name, avatarUrl)
-

Buyer Module

End-to-End Flow

1. POST /api/auth/register (buyer/default)
2. POST /api/auth/otp/verify (token issued)
3. optional POST /api/auth/setup (if setupRequired = true)
4. POST /api/auth/login
5. PATCH /api/auth/me for profile updates

1) Register Buyer

API: POST /api/auth/register

Request

```
{
  "email": "buyer@example.com",
  "password": "Buyer@1234",
  "role": "buyer",
  "name": "Buyer One",
  "city": "London"
}
```

Response (success shape)

```
{
  "error": false,
  "message": "Registration successful. Please verify your email with the OTP sent to your inbox.",
  "data": {
    "email": "buyer@example.com",
    "userId": 303,
    "otp": "445566",
    "challengeId": 99112,
    "expires-in": 600,
    "user": {}
  }
}
```

```
}
```

2) Verify OTP + Setup/Profile

OTP API: POST /api/auth/otp/verify

Setup API: POST /api/auth/setup (if needed)

Profile update API: PATCH /api/auth/me

Common Auth APIs (All Modules)

Login

API: POST /api/auth/login

Request

```
{
  "email": "user@example.com",
  "password": "Strong@1234",
  "deviceId": "optional-device-id"
}
```

Success response shape

```
{
  "error": false,
  "message": "You have logged in successfully.",
  "data": {
    "email": "user@example.com",
    "userId": 1,
    "accessToken": "eyJ...",
    "refreshToken": "eyJ...",
    "expires-at": "2026-04-17T12:30:00.000Z",
    "setupRequired": false,
    "accountStatus": "active",
    "emailStatus": "verified",
    "role": "buyer",
    "user": {}
  }
}
```

Get Auth Context / Application Status

API: GET /api/auth/me

Header: Authorization: Bearer <accessToken>

Response includes:

- setupRequired
- user
- guruApplication
- promoterApplication
- networkManagerApplication

Refresh / Logout

- POST /api/auth/refresh with { "refreshToken": "..."} }
- POST /api/auth/logout
- POST /api/auth/logout-all

Password / OTP Support

- POST /api/auth/forgot-password
- POST /api/auth/reset-password
- POST /api/auth/otp/resend

Guru Portal

Guru Folder API Documentation (Promoters, Referral, Credit/Wallet)

Base URL: `{{BASE_URL}}`

Primary module prefix: `{{BASE_URL}}/api/gurus`

Auth header for protected APIs: `Authorization: Bearer <accessToken>`

This document is compiled from current backend code in Guru-related routes/controllers and covers:

- Guru -> Promoters full module
- Guru referral flow
- Credit engine and wallet APIs used around Guru/Promoter economics

1) Guru -> Promoters Full Module

1.4 Activate pending promoter from dashboard

API Name: Activate Pending Promoter

Method/Path: `POST /api/gurus/dashboard/promoters/:promoterId/activate`

Auth: Guru token required

Purpose: Directly activate a pending promoter (supports invite-linked rows too).

Path params

- `promoterId` (required)

Success response shape

```
{
  "error": false,
  "message": "Promoter activated successfully.",
  "data": {
    "application": { "id": 1001, "accountStatus": "approved" },
    "user": {
      "userId": 321,
      "email": "john@example.com",
      "name": "John Doe",
      "role": "promoter",
      "accountStatus": "active",
      "rolesVersion": 6
    }
  }
}
```

1.5 List attached promoters

API Name: Get Attached Promoters

Method/Path: GET `/api/gurus/dashboard/promoters`

Auth: Active Guru required

Purpose: Return merged list of attached promoters + pending invite rows.

Query params

- `dateFrom`, `dateTo` (optional)
- `page` (default 1)
- `limit` (default 20, max 100)
- `search` (optional)

Success response shape (from code/Postman contract)

```
{
  "error": false,
  "data": {
    "quarterWindowUtc": {
      "startsAt": "2026-04-01T00:00:00.000Z",
      "endsAt": "2026-04-06T05:37:26.727Z"
    }
  }
}
```

```

    },
    "promoters": [
      {
        "promoterId": "637",
        "userNo": "637",
        "name": "fatima test promoter",
        "email": "gaweda7362@availors.com",
        "avatarUrl": null,
        "accountStatus": "active",
        "link": {
          "source": "referral_link",
          "attachedAt": "2026-04-02T00:45:28.096Z"
        },
        "referral": {
          "currentMode": "referral",
          "isReferralModeActive": true,
          "signedUpViaReferral": false,
          "fromReferralLink": true,
          "referralWindowExpiresAt": null
        },
        "credits": {
          "currency": "GBP",
          "guruCreditLedgerConfirmedGbp": 0,
          "guruCreditLedgerProjectedGbp": 0,
          "guruCommissionsRecordedTotalGbp": 0,
          "figmaCreditNowGbp": 0,
          "figmaCreditNormalGbp": 0
        },
        "stats": {
          "ticketsSold": 0,
          "grossSalesPence": 0,
          "grossSalesGbp": 0,
          "sprintContributionSettledTicketsThisUtcQuarter": 0,
          "refundsCountThisUtcQuarter": 0,
          "refundRatePercentThisUtcQuarter": 0,
          "eventsCount": 0
        },
        "ticketsSold": 0,
        "grossSales": 0,
        "joinedAt": "2026-04-02T00:45:28.096Z"
      }
    ]
  },
  "meta": {
    "requestId": "f83438e5-da07-4872-aa9e-27d34455c050",
    "timestamp": "2026-04-06T05:37:24.596Z"
  }
}

```

Important: response wrapper for this endpoint uses standard format:

- `error: false`
- `data: { ... }`
- `meta: { requestId, timestamp }`

`search` and `pagination` are present when query params are used; Postman saved responses may omit them depending on request/state.

1.6 Promoter deep-dive endpoints

- `GET /api/gurus/dashboard/promoters/:promoterId/details` -> profile header + summary cards
- `GET /api/gurus/dashboard/promoters/:promoterId/charts` -> chart series
(`granularity=daily|weekly|monthly`)
- `GET /api/gurus/dashboard/promoters/:promoterId/stats` -> stats
(`period=daily|weekly|monthly`)
- `GET /api/gurus/dashboard/promoters/:promoterId/history` -> history feed
- `GET /api/gurus/dashboard/promoters/:promoterId` -> performance/events summary

All require Guru ownership of that promoter (`requirePromoterOwnership`).

2) Guru Referral Flow

- Invite promoters referral link + live network stats

API Name: Invite Promoters Referral Link

Method/Path: `GET /api/gurus/promoters/invite/referral-link`

Also supports: `POST /api/gurus/promoters/invite/referral-link`

Auth: Guru token required

Purpose: Return referral link plus network counters for invite screen.

Success response shape

```
{
  "error": false,
  "data": {
    "referralCode": "ABC123",
    "referral_token": "ABC123",
    "referralLink": "https://frontend/register?referral_token=ABC123",
    "createdAt": "2026-04-17T10:00:00.000Z",
    "networkStats": {
      "promotersJoined": 7,
      "ticketsSold": 320,
      "earned": { "amount": "42.50", "currency": "GBP" }
    },
    "meta": {
      "statsLabel": "Live Stats",
      "footnote": "Promoters who sign up with this link join your network."
    }
  },
  "meta": {
    "requestId": "uuid",
    "timestamp": "2026-04-17T10:00:00.000Z"
  }
}
```

2.6 Referral-based promoter registration endpoint

API Name: Promoter Register via Referral

Method/Path: POST /api/auth/promoter/register

Auth: Public

Purpose: Register promoter using referral token flow.

Wallet Module

Base URL: {{BASE_URL}}

Auth: Authorization: Bearer <accessToken>

1) Get Wallet Summary

API Name: Wallet Me

Method/Path: GET /api/v1/wallet/me

Controller: getWalletMe

Service: getWalletMeForUser

Purpose

- Returns wallet summary for authenticated wallet role (promoter, guru, network_manager)
- Includes balances, service fee info, withdrawal eligibility
- For guru role, returns level-aware wallet behavior

Success Response (Guru example shape)

```
{  
  
  "wallet_id": "42",  
  
  "user_id": "123",  
  
  "role": "guru",  
  
  "balances": {  
  
    "projected": 9.3,  
  
    "confirmed": 0,  
  
    "net_withdrawable": 0,  
  
    "lifetime_earned": 0,  
  
    "sprint_credit": 0  
  
  },  
  
  "service_fee": {  
  
    "rate": 0.15,
```

```
    "deducted_this_month": 0,

    "deducted_lifetime": 0

  },

  "withdrawal_eligibility": {

    "eligible_percent": 0,

    "reason": "No cash withdrawal at Level 2 without enhanced licence. All confirmed credit held as sprint credit."

  },

  "retrieved_at": "2026-04-17T12:00:00.000Z",

  "level": 2,

  "territory_id": "1",

  "unlock_status": {

    "unlocked": true

  },

  "sprint": {

    "active": true,

    "tickets_in_window": 0,

    "tickets_required": 5000,

    "progress_percent": 0,

    "window_start": "2026-04-01T00:00:00.000Z",

    "window_expires": "2026-07-01T00:00:00.000Z",

    "days_remaining": 75

  }
```

```
}
```

Error Responses

401

```
{
```

```
  "error": "UNAUTHORIZED",
```

```
  "message": "Invalid or expired token"
```

```
}
```

403

```
{
```

```
  "error": "UNAUTHORIZED_ROLE",
```

```
  "message": "Kings Account and Finance use admin wallet endpoints"
```

```
}
```

404

```
{
```

```
  "error": "WALLET_NOT_FOUND",
```

```
  "message": "Wallet not initialised for this user"
```

```
}
```

500

```
{
```

```
  "error": "INTERNAL_ERROR",
```

```
"message": "Unable to load wallet"

}
```

2) Get Wallet Transactions

API Name: Wallet Transactions

Method/Path: GET /api/v1/wallet/transactions

Controller: getWalletTransactionsHandler

Service: getWalletTransactions

Purpose

- Returns paginated `credit_ledger` transactions for authenticated wallet role
- Supports status/date/event filters
- Includes page totals summary

Query Params

- `page` (optional, default 1)
- `limit` (optional, default 20, max 100)
- `status` (optional): `all` | `projected` | `confirmed` | `void`
- `from` (optional): YYYY-MM-DD
- `to` (optional): YYYY-MM-DD
- `event_id` (optional)

Success Response Shape

```
{

  "wallet_id": "42",

  "role": "guru",

  "pagination": {
```

```
    "page": 1,  
  
    "limit": 20,  
  
    "total_entries": 2,  
  
    "total_pages": 1  
  },  
  
  "summary": {  
  
    "total_projected_in_view": 9.3,  
  
    "total_confirmed_in_view": 0,  
  
    "total_void_in_view": 0  
  },  
  
  "transactions": [  
  
    {  
  
      "entry_id": "991",  
  
      "entry_type": "CREDIT_PROJECTED",  
  
      "status": "PROJECTED",  
  
      "amount": 9.3,  
  
      "role": "guru",  
  
      "tier_number": 2,  
  
      "event_id": "5001",  
  
      "event_title": "Event Title",  
  
      "ticket_id": "80001",  
  
      "reference_note": null,
```

```
    "created_at": "2026-04-17T10:00:00.000Z"

  }

],

  "retrieved_at": "2026-04-17T12:00:00.000Z"

}
```

Transaction entry_type values returned by API mapper

- CREDIT_PROJECTED
- CREDIT_CONFIRMED
- CREDIT_VOID_REFUND
- CREDIT_VOID_CANCEL
- REFERRAL_BONUS
- SERVICE_FEE_DEDUCTION
- WITHDRAWAL

Error Responses

401

```
{

  "error": "UNAUTHORIZED",

  "message": "Invalid or expired token"

}
```

403

```
{

  "error": "UNAUTHORIZED_ROLE",

  "message": "Kings Account and Finance use admin wallet endpoints"

}
```

```
}
```

403 (no wallet for role)

```
{
```

```
  "error": "UNAUTHORIZED_ROLE",
```

```
  "message": "No wallet for this role"
```

```
}
```

400 invalid page

```
{
```

```
  "error": "INVALID_PAGE",
```

```
  "message": "Page must be a positive integer"
```

```
}
```

400 invalid date range

```
{
```

```
  "error": "INVALID_DATE_RANGE",
```

```
  "message": "from must be before to"
```

```
}
```

500

```
{
```

```
  "error": "INTERNAL_ERROR",
```

```
  "message": "Unable to load transactions"
```

}

Credit Engine and Settings APIs

Base URL: `{{BASE_URL}}`

Auth header (all protected APIs): `Authorization: Bearer <accessToken>`

This doc covers only:

- Credit Engine APIs
 - Settings APIs
- for Guru module usage.

A) Credit Engine (Guru)

1) Get Credit Tier Table / Match Tier

API Name: Credit Tier

Method/Path: `GET /api/v1/credit/tier`

Auth: required

Controller: `getCreditTier`

Purpose

- Returns booking-fee tier table and split info
- If `ticket_price` is sent, returns matched tier only

Query Params

- `ticket_price` (optional, number > 0)
- `territory_id` (optional)
- `page` (optional)
- `limit` (optional)

Response (matched tier example)

```
{  
  
  "territory_id": "1",  
  
  "territory_name": "London",  
  
  "matched_tier": {  
  
    "tier_number": 2,  
  
    "price_range": { "min": "34.35", "max": "34.35" },  
  
    "booking_fee": "1.60",  
  
    "vat_amount": "0.40",  
  
    "noda_fee": "0.35",  
  
    "distributable_pool": "1.60",  
  
    "splits": {  
  
      "promoter": "0.50",  
  
      "guru": "0.35",  
  
      "network_manager": "0.00",  
  
      "eventopia": "0.40",  
  
      "reinvestment": "0.00"  
  
    }  
  }  
}
```

```
    },  
  
    "referral_mode_active": false,  
  
    "retrieved_at": "2026-04-17T12:00:00.000Z"  
}
```

Common Errors

- 400 INVALID_TICKET_PRICE
- 400 INVALID_PAGE
- 400 INVALID_LIMIT
- 404 TERRITORY_NOT_FOUND
- 500 SERVER_ERROR

B) Settings (Guru)

The settings routes are mounted at `/api/v1`.

1) Get Profile

API Name: Get Profile

Method/Path: GET `/api/v1/profile`

Auth: required

Allowed roles: promoter, guru, network_manager, buyer

Response

```
{  
  
    "error": false,  
  
    "message": "Profile retrieved successfully",  
  
    "data": {
```

```
    "id": 123,  
  
    "name": "Guru Name",  
  
    "email": "guru@example.com",  
  
    "phone": "+447911123456",  
  
    "role": "guru",  
  
    "avatar_url": null,  
  
    "territory": "London"  
  
  }  
  
}
```

2) Update Profile

API Name: Update Profile

Method/Path: PUT /api/v1/profile

Auth: required

Request Body

```
{  
  
  "name": "Updated Guru Name",  
  
  "email": "newguru@example.com",  
  
  "phone": "+447911123456",  
  
  "avatar_url": "https://cdn.example/avatar.png",  
  
  "current_password": "Current@123"  
  
}
```

Notes

- `territory` is read-only (if sent -> rejected)
- If changing `email`, `current_password` is required
- On email change, system sends OTP and marks email pending

Success Response

```
{  
  
  "error": false,  
  
  "message": "Profile updated successfully",  
  
  "data": {  
  
    "success": true,  
  
    "email_verification_sent": true,  
  
    "challengeId": 99881,  
  
    "expiresIn": 600  
  
  }  
}
```

Common Errors

- 400 no fields / missing current password for email change
- 401 invalid current password
- 409 email already in use
- 422 invalid phone or territory update attempt

3) Verify Changed Email

API Name: Verify Email

Method/Path: POST /api/v1/verify-email

Auth: required

Purpose: verify OTP sent after email change

Request Body

```
{  
  
  "email": "newguru@example.com",  
  
  "otp_code": "123456",  
  
  "challenge_id": "99881"  
  
}
```

Success Response

```
{  
  
  "error": false,  
  
  "message": "Email verified successfully. Your profile has been updated.",  
  
  "data": {  
  
    "success": true,  
  
    "email_verified": true,  
  
    "email": "newguru@example.com"  
  
  }  
  
}
```

4) Change Password

API Name: Change Password

Method/Path: POST /api/v1/auth/change-password

Auth: required

Controller: changePasswordV1

Request Body

```
{  
  
  "current_password": "Current@123",  
  
  "new_password": "NewStrong@123",  
  
  "confirm_new_password": "NewStrong@123"  
  
}
```

Success Response (shape)

```
{  
  
  "error": false,  
  
  "message": "Password updated successfully. Other sessions have been  
signed out.",  
  
  "data": {  
  
    "success": true  
  
  }  
  
}
```

Promoter Portal

Promoter APIs Documentation (Events, Referral, Escrow, Credit, Wallet, Settings)

Base URL: `{{BASE_URL}}`

Auth header for protected APIs: `Authorization: Bearer <accessToken>`

This documentation is consolidated from current backend code
(routes/controllers/services) and existing module docs.

Primary route mounts used here:

- `/api/promoters/*` from `src/routes/promoters.routes.js`
- `/api/v1/credit/*` from `src/routes/credit.routes.js`
- `/api/v1/wallet/*` from `src/routes/wallet.routes.js`
- `/api/v1/escrow/*` from `src/routes/escrow.routes.js`
- `/api/v1/* settings` from `src/routes/settings.routes.js`

1) Promoter Events APIs

Module prefix: `/api/promoters`

1.1 Create Event

Endpoint: `POST /api/promoters/events`

Purpose: Create draft event for authenticated active promoter.

Request body (example):

```
{  
  "title": "Summer Night Show",  
  "description": "Live concert event",  
  "startAt": "2026-07-20T18:00:00.000Z",  
  "endAt": "2026-07-20T22:00:00.000Z",  
}
```

```
"timezone": "Europe/London",
"format": "in_person",
"accessMode": "ticketed",
"visibilityMode": "public",
"city": "London",
"venueName": "Royal Arena",
"venueAddress": "221B Baker Street",
"categoryId": 3,
"tagIds": [1, 4]
}
```

Response example:

```
{
  "error": false,
  "data": {
    "id": 127,
    "message": "Event created successfully"
  },
  "meta": {
    "requestId": "req_123",
    "timestamp": "2026-04-17T12:00:00.000Z"
  }
}
```

1.2 List Promoter Events

Endpoint: GET /api/promoters/events

Purpose: Get promoter-owned events list.

Response example (shape):

```
{
  "error": false,
  "data": {
    "items": [
      {
        "id": 127,
        "title": "Summer Night Show",
        "status": "draft",
        "startAt": "2026-07-20T18:00:00.000Z",
        "city": "London"
      }
    ],
    "pagination": {
      "page": 1,
      "pageSize": 20,

```

```
    "total": 1
  }
}
```

1.3 Event Detail

Endpoint: GET /api/promoters/events/:eventId

Purpose: Get full event detail (owner-only).

1.4 Update Event

Endpoint: PATCH /api/promoters/events/:eventId

Purpose: Update editable event fields.

Request body (example):

```
{
  "title": "Summer Night Show - Updated",
  "city": "Manchester",
  "visibilityMode": "private_link",
  "resetShareToken": true
}
```

Response example:

```
{
  "error": false,
  "data": {
    "id": 127
  }
}
```

1.5 Delete Event

Endpoint: DELETE /api/promoters/events/:eventId

Purpose: Delete event (owner-only).

1.6 Event Media / Category / Tags

- POST /api/promoters/events/:eventId/images (multipart field: image, query type=cover|gallery)
- DELETE /api/promoters/events/:eventId/cover

- DELETE /api/promoters/events/:eventId/gallery/:imageId
- PATCH /api/promoters/events/:eventId/images/reorder
- PUT /api/promoters/events/:eventId/category
- PUT /api/promoters/events/:eventId/tags

Reorder body example:

```
{
  "imageOrder": [19, 17, 23]
}
```

1.7 Event Status Lifecycle

- POST /api/promoters/events/:eventId/submit
- POST /api/promoters/events/:eventId/publish
- POST /api/promoters/events/:eventId/pause
- POST /api/promoters/events/:eventId/cancel
- POST /api/promoters/events/:eventId/republish
- POST /api/promoters/events/:eventId/complete

Publish success example:

```
{
  "error": false,
  "data": {
    "id": 127,
    "status": "published"
  }
}
```

1.8 Ticket Types (Promoter Event)

- GET /api/promoters/events/:eventId/ticket-types
- POST /api/promoters/events/:eventId/ticket-types
- PATCH /api/promoters/ticket-types/:ticketTypeId
- POST /api/promoters/ticket-types/:ticketTypeId/duplicate
- POST /api/promoters/ticket-types/:ticketTypeId/pause
- POST /api/promoters/ticket-types/:ticketTypeId/resume
- DELETE /api/promoters/ticket-types/:ticketTypeId

Create ticket type body example:

```
{
```

```
"name": "General Admission",  
"priceAmount": 1050,  
"bookingFeeAmount": 235,  
"capacityTotal": 100,  
"perOrderLimit": 10  
}
```

2) Promoter Referral APIs

Module prefix: `/api/promoters`

2.1 Get/Create Referral Link

Endpoint: `GET /api/promoters/referral/link`

Purpose: Returns reusable active shareable referral link for logged-in active promoter.

Response example (from controller/service shape):

```
{  
  "error": false,  
  "message": "Referral link retrieved successfully.",  
  "data": {  
    "referralId": 555,  
    "token": "a8f9d8...hex",  
    "status": "link_issued",  
    "referralLink":  
"https://frontend.example/register?referral_token=a8f9d8...hex",  
    "createdAt": "2026-04-17T12:00:00.000Z",  
    "source": "existing"  
  }  
}
```

2.2 List Referrals

Endpoint: `GET /api/promoters/referrals?page=1&limit=10&search=`

Purpose: List referral records as referrer and as referred, with pagination.

Response example:

```
{
```

```
"error": false,
"message": "Referrals retrieved successfully.",
"data": {
  "asReferrer": [
    {
      "id": 555,
      "status": "active",
      "ticket_count": 210,
      "referred_name": "John Doe",
      "referred_email": "john@example.com",
      "days_remaining": 67
    }
  ],
  "asReferred": [],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1,
    "hasNextPage": false,
    "hasPrevPage": false,
    "search": ""
  }
}
```

3) Escrow APIs

Module prefix: /api/v1/escrow

3.1 Coverage Ratio by Territory

Endpoint: GET /api/v1/escrow/coverage/:territory_id

Roles: finance, kings_account

Purpose: Return live coverage ratio for selected territory.

Success response (shape):

```
{
  "territory_id": 1,
  "coverage_ratio": 1.13,
  "liability_amount": 54000,
  "escrow_balance": 61000,
```

```
"retrieved_at": "2026-04-17T12:00:00.000Z"
}
```

Validation error example:

```
{
  "error": "INVALID_INPUT",
  "message": "territory_id must be a valid integer"
}
```

3.2 Promoter Escrow View

Endpoint: GET /api/v1/escrow/promoter/finance/escrow

Roles: promoter, finance, kings_account

Purpose: Promoter escrow overview (promoter sees only own escrow;
finance/kings can query by promoter_id).

Optional query: promoter_id (finance/kings only)

Response shape example:

```
{
  "promoter_id": 637,
  "projected_amount": 120.5,
  "confirmed_amount": 43.2,
  "payout_status": "pending",
  "entries": []
}
```

3.3 Interest History by Territory

Endpoint: GET

/api/v1/escrow/interest/:territory_id?from=YYYY-MM-DD&to=YYYY-MM-DD

Roles: finance, kings_account

Purpose: Interest accrual history and filtering by date range.

Date format error example:

```
{
  "error": "INVALID_DATE_FORMAT",
  "message": "from date must be in YYYY-MM-DD format"
}
```

4) Credit Engine APIs

Module prefix: `/api/v1/credit`

4.1 Credit Tier Table / Tier Match

Endpoint: `GET /api/v1/credit/tier`

Purpose: Return tier table with booking fee + split breakdown.

If `ticket_price` is provided, returns `matched_tier`.

Query params:

- `ticket_price` (optional)
- `territory_id` (optional)
- `page` (optional)
- `limit` (optional)

Matched tier response example:

```
{
  "territory_id": "1",
  "territory_name": "London",
  "matched_tier": {
    "tier_number": 2,
    "price_range": { "min": "34.35", "max": "34.35" },
    "booking_fee": "1.60",
    "vat_amount": "0.40",
    "noda_fee": "0.35",
    "distributable_pool": "1.60",
    "splits": {
      "promoter": "0.50",
      "guru": "0.35",
      "network_manager": "0.00",
      "eventopia": "0.40",
      "reinvestment": "0.00"
    }
  },
  "referral_mode_active": false,
  "retrieved_at": "2026-04-17T12:00:00.000Z"
}
```

4.2 Create Promoter Credit Wallet (Internal)

Endpoint: POST /api/v1/credit/wallets/create

Role: kings_account

Purpose: Create promoter credit wallet row.

Request body:

```
{
  "promoter_id": 637
}
```

Success response example:

```
{
  "wallet_id": "101",
  "promoter_id": "637",
  "currency": "GBP",
  "created_at": "2026-04-17T12:00:00.000Z"
}
```

4.3 Get Promoter Credit Wallet

Endpoint: GET /api/v1/credit/wallets/:promoter_id

Role requirement: authenticated promoter owning this :promoter_id.

Success response example:

```
{
  "wallet_id": "101",
  "promoter_id": "637",
  "projected_balance": 48.7,
  "confirmed_balance": 12.4,
  "lifetime_earned": 89.6,
  "currency": "GBP",
  "updated_at": "2026-04-17T12:00:00.000Z"
}
```

Forbidden example:

```
{
  "error": "Forbidden",
  "reason": "Authenticated user does not own this wallet"
}
```

5) Wallet APIs

Module prefix: `/api/v1/wallet`

5.1 Wallet Summary

Endpoint: `GET /api/v1/wallet/me`

Purpose: Return role-based wallet summary for authenticated user.

Success response example:

```
{
  "wallet_id": "42",
  "user_id": "637",
  "role": "promoter",
  "balances": {
    "projected": 12.5,
    "confirmed": 5.2,
    "net_withdrawable": 3.7,
    "lifetime_earned": 22.1,
    "sprint_credit": 0
  },
  "service_fee": {
    "rate": 0.15,
    "deducted_this_month": 0,
    "deducted_lifetime": 0
  },
  "withdrawal_eligibility": {
    "eligible_percent": 100,
    "reason": null
  },
  "retrieved_at": "2026-04-17T12:00:00.000Z"
}
```

5.2 Wallet Transactions

Endpoint: `GET /api/v1/wallet/transactions`

Purpose: Paginated transaction ledger list.

Query params:

- page (default 1)
- limit (default 20, max 100)
- status (all|projected|confirmed|void)
- from (YYYY-MM-DD)
- to (YYYY-MM-DD)
- event_id (optional)

Success response example:

```
{
  "wallet_id": "42",
  "role": "promoter",
  "pagination": {
    "page": 1,
    "limit": 20,
    "total_entries": 2,
    "total_pages": 1
  },
  "summary": {
    "total_projected_in_view": 9.3,
    "total_confirmed_in_view": 0,
    "total_void_in_view": 0
  },
  "transactions": [
    {
      "entry_id": "991",
      "entry_type": "CREDIT_PROJECTED",
      "status": "PROJECTED",
      "amount": 9.3,
      "event_id": "5001",
      "event_title": "Event Title",
      "created_at": "2026-04-17T10:00:00.000Z"
    }
  ],
  "retrieved_at": "2026-04-17T12:00:00.000Z"
}
```

6) Settings APIs (Promoter relevant + shared)

Module prefix: /api/v1

6.1 Get Profile

Endpoint: GET /api/v1/profile

Purpose: Get current user profile (promoter/guru/network_manager/buyer).

Response example:

```
{
  "error": false,
  "message": "Profile retrieved successfully",
  "data": {
    "id": 637,
    "name": "Fatima Promoter",
    "email": "fatima@example.com",
    "phone": "+447911123456",
    "role": "promoter",
    "avatar_url": null,
    "territory": "London"
  }
}
```

6.2 Update Profile

Endpoint: PUT /api/v1/profile

Purpose: Update profile fields; email change requires current password.

Request body example:

```
{
  "name": "Fatima Promoter Updated",
  "phone": "+447922334455",
  "email": "fatima.new@example.com",
  "current_password": "Current@123"
}
```

Success response example:

```
{
  "error": false,
  "message": "Profile updated successfully",
  "data": {
    "success": true,
    "email_verification_sent": true,
    "challengeId": 99881,
    "expiresIn": 600
  }
}
```

```
}  
}
```

6.3 Verify Changed Email

Endpoint: POST /api/v1/verify-email

Purpose: Verify OTP for changed email.

Request body:

```
{  
  "email": "fatima.new@example.com",  
  "otp_code": "123456",  
  "challenge_id": "99881"  
}
```

Success response:

```
{  
  "error": false,  
  "message": "Email verified successfully. Your profile has been updated.",  
  "data": {  
    "success": true,  
    "email_verified": true,  
    "email": "fatima.new@example.com"  
  }  
}
```

6.4 Change Password

Endpoint: POST /api/v1/auth/change-password

Purpose: Change password for authenticated user.

Request body:

```
{  
  "current_password": "Current@123",  
  "new_password": "NewStrong@123",  
  "confirm_new_password": "NewStrong@123"  
}
```

Buyer Portal



Buyer APIs Documentation (Events, Orders/Ticket Purchase, Refund Center, Settings)

Base URL: `{{BASE_URL}}`

Auth header (protected APIs): `Authorization: Bearer <accessToken>`

This document is compiled from current backend route/controller code.

Primary mounts used:

- Public events: `/api/events/*`
- Orders + buyer ticket/refund flow: `/api/orders/*`
- Buyer alias routes: `/api/buyer/*` (via module aliases)
- Settings: `/api/v1/*`

1) Buyer Event APIs

Module prefix: `/api/events`

1.1 List Public Events

Endpoint: `GET /api/events`

Purpose: Browse buyer-visible public events.

Query params:

- `city` (optional)
- `categoryId` (optional)
- `tagIds` (optional, csv or array)
- `dateFrom, dateTo` (optional)
- `search` (optional)
- `sort` = `soonest` | `newest` | `popular` (default `soonest`)

- page, pageSize

Response example:

```
{
  "error": false,
  "data": {
    "items": [
      {
        "id": 127,
        "title": "Summer Night Show",
        "startAt": "2026-07-20T18:00:00.000Z",
        "endAt": "2026-07-20T22:00:00.000Z",
        "city": "London",
        "venueName": "Royal Arena",
        "format": "in_person",
        "accessMode": "ticketed",
        "category": { "id": 3, "name": "Music", "slug": "music" },
        "tags": [{ "id": 1, "name": "Live", "slug": "live" }],
        "coverImageUrl": "/uploads/event-cover.jpg",
        "ticketsSold": 74
      }
    ],
    "pagination": {
      "page": 1,
      "pageSize": 20,
      "total": 1
    }
  },
  "meta": {
    "requestId": "req_123",
    "timestamp": "2026-04-17T12:00:00.000Z"
  }
}
```

1.2 Get Public Event Detail

Endpoint: GET /api/events/:id

Purpose: Full buyer event detail for public+buyer-visible events.

Response example:

```
{
  "error": false,
  "data": {
    "event": {
      "id": 127,
```

```

    "title": "Summer Night Show",
    "description": "Live concert event",
    "startAt": "2026-07-20T18:00:00.000Z",
    "endAt": "2026-07-20T22:00:00.000Z",
    "timezone": "Europe/London",
    "city": "London",
    "venueName": "Royal Arena",
    "venueAddress": "221B Baker Street",
    "format": "in_person",
    "accessMode": "ticketed",
    "category": { "id": 3, "name": "Music", "slug": "music" },
    "tags": [{ "id": 1, "name": "Live", "slug": "live" }],
    "promoter": { "name": "Fatima Promoter" },
    "images": {
      "coverImageUrl": "/uploads/event-cover.jpg",
      "galleryImageUrls": ["/uploads/event-1.jpg"]
    }
  },
  "ticketTypes": [
    {
      "id": 501,
      "name": "General Admission",
      "currency": "GBP",
      "priceAmount": 1050,
      "bookingFeeAmount": 235,
      "totalAmount": 1285,
      "capacityTotal": 100,
      "capacitySold": 74,
      "capacityRemaining": 26,
      "perOrderLimit": 10,
      "status": "active"
    }
  ]
}

```

2) Buyer Ticket Purchase + Order Flow APIs

Primary module prefix: `/api/orders`

2.1 Create Order (Step 1)

Endpoint: POST /api/orders

Role: buyer

Purpose: Create pending order + reserve ticket inventory + return payment reference/checkout URL.

Request body:

```
{
  "event_id": 127,
  "idempotency_key": "0eb7d27e-f7f1-4e71-878a-640c253fb1ce",
  "items": [
    {
      "ticket_tier_id": 501,
      "quantity": 2,
      "attendees": [
        { "name": "Fatima A" },
        { "name": "Fatima B" }
      ]
    }
  ]
}
```

Success response example:

```
{
  "error": false,
  "data": {
    "order": {
      "id": 9981,
      "status": "pending",
      "total_ticket_amount": 21,
      "total_booking_fee": 4.7,
      "grand_total": 25.7,
      "expires_at": "2026-04-17T12:15:00.000Z",
      "payment_intent_ref": "pi_3N...",
      "items": [
        {
          "attendee_name": "Fatima A",
          "tier_name": "General Admission",
          "ticket_price": 10.5,
          "booking_fee": 2.35
        }
      ],
      "stripe_checkout_url": "https://checkout.stripe.com/...",
      "stripe_checkout_session_id": "cs_test..."
    }
  }
}
```

```
}  
}
```

Common errors:

- 400 `VALIDATION_ERROR` (missing fields, invalid IDs, attendee mismatch)
- 409 `QUANTITY_EXCEEDED` (not enough tickets)
- 503 `PAYMENT_SETUP_FAILED` (payment setup could not be created)

2.2 Confirm Order (Step 2)

Endpoint: `POST /api/orders/:id/confirm`

Purpose: Confirm payment result; on success mints tickets + finalizes ledgers.

Request body:

```
{  
  "payment_ref": "pi_3N...",  
  "payment_status": "success"  
}
```

Success response example:

```
{  
  "error": false,  
  "data": {  
    "order": {  
      "id": 9981,  
      "status": "completed",  
      "items": [  
        {  
          "ticket_id": 7001,  
          "attendee_name": "Fatima A",  
          "tier_name": "General Admission"  
        }  
      ]  
    },  
    "escrow_entry_created": true,  
    "confirmation_email_sent": false  
  }  
}
```

2.3 Get Buyer Tickets (Ticket Wallet View)

Endpoints:

- **Canonical:** GET /api/orders/buyer/tickets
- **Alias:** GET /api/buyer/tickets

Purpose: Return buyer tickets grouped by event+order.

Query params: status=active|used|cancelled, page, limit

Response example:

```
{
  "error": false,
  "data": {
    "data": [
      {
        "event": {
          "id": 127,
          "title": "Summer Night Show",
          "date": "2026-07-20T18:00:00.000Z",
          "start_time": "18:00",
          "venue_name": "Royal Arena",
          "status": "published"
        },
        "order": {
          "id": 9981,
          "created_at": "2026-04-17T12:00:00.000Z",
          "total_ticket_amount": 21,
          "total_booking_fee": 4.7,
          "grand_total": 25.7
        },
        "tickets": [
          {
            "id": 7001,
            "attendee_name": "Fatima A",
            "tier_name": "General Admission",
            "ticket_price": 10.5,
            "booking_fee": 2.35,
            "status": "active",
            "qr_code_url": "/api/buyer/tickets/7001/qr"
          }
        ]
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 1,
      "total_pages": 1
    }
  }
}
```

```
}  
}  
}
```

2.4 Get Buyer Ticket QR

Endpoints:

- **Canonical:** GET /api/orders/buyer/tickets/:itemId/qr
- **Alias:** GET /api/buyer/tickets/:itemId/qr

Purpose: Get QR payload/hash/image for a specific buyer-owned ticket.

Response example:

```
{  
  "error": false,  
  "data": {  
    "ticket_item_id": 7001,  
    "qr_code_hash": "b5d8f3...",  
    "qr_payload": {  
      "ticket_item_id": 7001,  
      "order_id": 9981,  
      "event_id": 127,  
      "buyer_id": 900,  
      "attendee_name": "Fatima A",  
      "tier_name": "General Admission"  
    },  
    "qr_image_base64": "...",  
    "status": "active"  
  }  
}
```

3) Buyer Refund Center APIs

Primary module prefix: /api/orders/buyer

3.1 Refund Center List (Cancelled Events Tickets)

Endpoint: GET /api/orders/buyer/tickets/cancelled-events

Role: buyer

Purpose: Show buyer's tickets for cancelled events and refund window eligibility.

Query params: page, limit

Response example:

```
{
  "error": false,
  "data": {
    "items": [
      {
        "event": {
          "id": 127,
          "title": "Summer Night Show",
          "date": "2026-07-20T18:00:00.000Z",
          "venue_name": "Royal Arena",
          "city": "London",
          "status": "cancelled",
          "cancelled_at": "2026-07-10T11:00:00.000Z",
          "cancel_reason": "Operational reason"
        },
        "ticket": {
          "id": 7001,
          "order_item_id": 8201,
          "order_id": 9981,
          "order_number": "EVT-171...",
          "attendee_name": "Fatima A",
          "tier_name": "General Admission",
          "ticket_price": 10.5,
          "booking_fee": 2.35,
          "status": "active",
          "purchased_at": "2026-04-17T12:00:00.000Z"
        },
        "refund_window": {
          "stage": "primary",
          "primary_window_days": 30,
          "primary_window_ends_at": "2026-08-09T11:00:00.000Z",
          "days_left_in_current_window": 22,
          "can_request_refund": true
        }
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 1,
      "total_pages": 1
    }
  }
}
```

3.2 Submit Refund Request

Endpoint: POST /api/orders/buyer/refunds

Role: buyer

Purpose: Submit refund case for cancelled event ticket inside refund window; backend ring-fences escrow liability.

Request body:

```
{
  "order_item_id": 8201,
  "reason_code": "EVENT_CANCELLED"
}
```

Success response example:

```
{
  "error": false,
  "data": {
    "refund_case": {
      "id": 301,
      "order_item_id": 8201,
      "reason_code": "EVENT_CANCELLED",
      "status": "submitted",
      "amount": 21,
      "escrow_ring_fenced": true,
      "submitted_at": "2026-04-17T12:05:00.000Z"
    },
    "message": "Refund request submitted successfully"
  }
}
```

Common errors:

- 400 VALIDATION_ERROR (missing/invalid body fields)
 - 400 INVALID_STATE (event not cancelled, unpaid order, or no refund window)
 - 400 REFUND_WINDOW_CLOSED
 - 403 FORBIDDEN (order item not owned by buyer)
 - 409 DUPLICATE_REFUND_CASE
 - 409 INSUFFICIENT_ESCROW
-

4) Buyer Settings APIs

Module prefix: /api/v1

4.1 Get Profile

Endpoint: GET /api/v1/profile

Purpose: Fetch current buyer profile.

Response example:

```
{
  "error": false,
  "message": "Profile retrieved successfully",
  "data": {
    "id": 900,
    "name": "Buyer Name",
    "email": "buyer@example.com",
    "phone": "+447911123456",
    "role": "buyer",
    "avatar_url": null,
    "territory": "London"
  }
}
```

4.2 Update Profile

Endpoint: PUT /api/v1/profile

Purpose: Update buyer name/email/phone/avatar.

Request body:

```
{
  "name": "Buyer Updated",
  "email": "buyer.new@example.com",
  "phone": "+447922334455",
  "current_password": "Current@123"
}
```

Success response example:

```
{
  "error": false,
```

```
"message": "Profile updated successfully",
"data": {
  "success": true,
  "email_verification_sent": true,
  "challengeId": 88771,
  "expiresIn": 600
}
```

4.3 Verify Changed Email

Endpoint: POST /api/v1/verify-email

Purpose: Verify OTP for changed email.

Request body:

```
{
  "email": "buyer.new@example.com",
  "otp_code": "123456",
  "challenge_id": "88771"
}
```

4.4 Change Password

Endpoint: POST /api/v1/auth/change-password

Purpose: Change buyer password.

Request body:

```
{
  "current_password": "Current@123",
  "new_password": "NewStrong@123",
  "confirm_new_password": "NewStrong@123"
}
```
